



Programação Orientada a Objetos

Herança e Polimorfismo em Python

Prof. Edson José Pacheco

2026

Fundamentos de Herança

- ▶ Compreender a relação "é-um" entre classes
- ▶ Criar hierarquias de classes usando herança
- ▶ Utilizar `super()` para métodos da base
- ▶ Classe concreta, abstrata e interface

Polimorfismo

- ▶ Aplicar sobrescrita (*override*) de métodos
- ▶ Escrever código genérico usando abstrações
- ▶ Implementar o padrão *State* com herança

Herança Múltipla

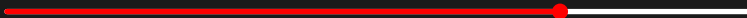
- ▶ Entender o MRO (*Method Resolution Order*)
- ▶ Criar e utilizar *Mixins* de forma segura
- ▶ Evitar o problema do diamante

Decoradores Python

- ▶ `@property` para encapsulamento
- ▶ `@classmethod` para *factory methods*
- ▶ `@staticmethod` para utilitários
- ▶ `@abstractmethod` em classes abstratas



CLIQUE PARA ABRIR NO NAVEGADOR



 Um *Overview* sobre Herança

Motivação

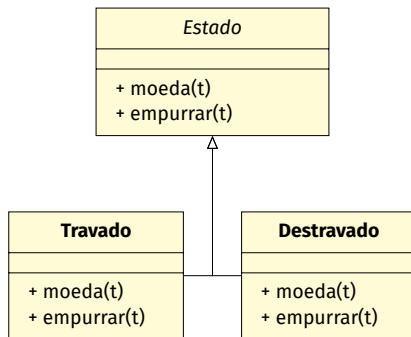


Observe o padrão **State** que implementamos:

- ▶ Travado e Destravado compartilham a mesma **interface**
- ▶ Ambos implementam `moeda()` e `empurrar()`
- ▶ O Torniquete não precisa saber qual estado concreto está ativo

Problema: Como garantir que todos os estados implementem os mesmos métodos?

Solução: Herança — definir um contrato comum.



Reutilização de Código

- ▶ Atributos e métodos comuns definidos uma vez na classe base
- ▶ Subclasses herdam automaticamente
- ▶ Evita duplicação (DRY)

Extensibilidade

- ▶ Novas classes podem ser adicionadas sem modificar código existente
- ▶ Princípio Open/Closed

Polimorfismo

- ▶ Objetos de diferentes classes tratados uniformemente
- ▶ Código cliente depende de abstrações
- ▶ Facilita testes e manutenção

Organização Hierárquica

- ▶ Taxonomia natural do domínio
- ▶ “É-um” (is-a relationship)

Nem toda relação entre classes deve usar herança. A escolha errada gera hierarquias frágeis.

Herança (“é um”)

Python

```
class Sensor(Dispositivo):  
    """Sensor E UM Dispositivo"""  
    def ler(self):  
        return self.registrador.valor
```

- ▶ Subclasse **é uma especialização** da base
- ▶ Herda interface e implementação
- ▶ Acoplamento forte: mudanças na base propagam

Composição (“tem um”)

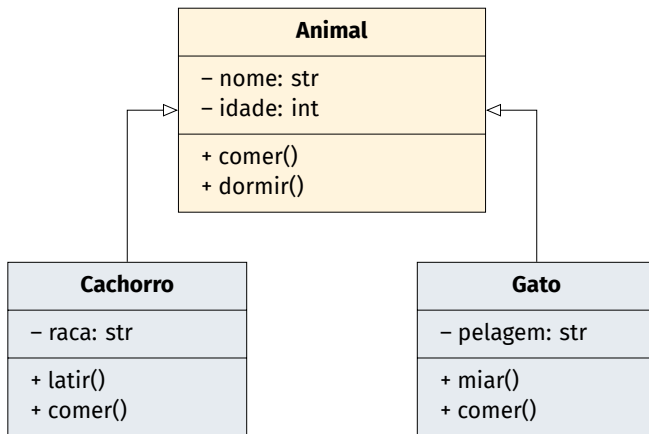
Python

```
class SCADA:  
    """SCADA TEM UM gateway e UM  
    historian"""  
    def __init__(self):  
        self.gateway = GatewayIEC61850()  
        self.historyan = Historian()
```

- ▶ Objeto **contém** outros objetos
- ▶ Delega tarefas aos componentes
- ▶ Acoplamento fraco: troca componentes facilmente

Conceitos Fundamentais





- ▶ **Superclasse / Classe Base / Classe Pai:** `Animal`
- ▶ **Subclasse / Classe Derivada / Classe Filha:** `Cachorro`, `Gato`
- ▶ **Herdar:** Receber atributos e métodos da superclasse
- ▶ **Sobrescrever (Override):** Redefinir método herdado (`comer()`)

```
class Animal:
    def __init__(self, nome: str):
        self.nome = nome

    def comer(self):
        print(f"{self.nome} está comendo.")

    def dormir(self):
        print(f"{self.nome} está dormindo.")

class Cachorro(Animal): # Herda de Animal
    def __init__(self, nome: str, raca: str):
        super().__init__(nome) # Chama construtor
        da base
        self.raca = raca

    def latir(self):
        print(f"{self.nome} faz Au Au!")

    def comer(self): # Sobrescreve
        print(f"{self.nome} devora a ração!")
```

Pontos-chave:

- ▶ Herança: class Filha(Pai)
- ▶ super() acessa a classe base
- ▶ super().__init__() chama construtor do pai
- ▶ Métodos podem ser **sobrescritos** (override)
- ▶ Métodos novos podem ser **adicionados**

```
class Cachorro(Animal):  
    def __init__(self, nome: str, raca: str):  
        # CORRETO: usa super()  
        super().__init__(nome)  
        self.raca = raca  
  
    def comer(self):  
        # Chama implementação da base primeiro  
        super().comer()  
        # Adiciona comportamento específico  
        print("E pede mais!")  
  
    def apresentar(self):  
        # Acessa atributo herdado  
        print(f"Sou {self.nome}, um {self.raca}")  
)
```

Formas de chamar a base:

1. Recomendado:

- ▶ `super().metodo()`
- ▶ Funciona com herança múltipla

2. Explícito (evitar):

- ▶ `Animal.metodo(self)`
- ▶ Quebra com herança múltipla
- ▶ Acopla ao nome da classe

Tipos de Classes



Classe Concreta	Classe Abstrata	Interface
Pode ser instanciada	Não pode ser instanciada	Não pode ser instanciada
Todos os métodos implementados	Pode ter métodos abstratos e concretos	Apenas métodos abstratos
Representa objeto real	Representa conceito parcial	Define contrato puro
Cachorro, Gato	Animal	Comparable, Iterator

Em Python: Não há palavra-chave interface. Usamos classes abstratas para ambos os propósitos.

```
from abc import ABC, abstractmethod

class Forma(ABC):
    """Classe abstrata - não pode ser instanciada"""

    def __init__(self, cor: str):
        self.cor = cor # Atributo concreto

    @abstractmethod
    def area(self) -> float:
        """Método abstrato - DEVE ser implementado"""
        pass

    @abstractmethod
    def perimetro(self) -> float:
        pass

    def descricao(self) -> str:
        """Método concreto - pode ser herdado"""
        return f"Forma de cor {self.cor}"
```

Componentes:

- ▶ ABC: Abstract Base Class
- ▶ @abstractmethod: marca método abstrato

Regras:

- ▶ Herdar de ABC
- ▶ Métodos com @abstractmethod não têm corpo útil
- ▶ Subclasses concretas **devem** implementar todos os métodos abstratos
- ▶ Pode ter métodos concretos também

```
import math

class Circulo(Forma):
    def __init__(self, cor: str, raio: float):
        super().__init__(cor)
        self.raio = raio

    def area(self) -> float:
        return math.pi * self.raio ** 2

    def perimetro(self) -> float:
        return 2 * math.pi * self.raio

class Retangulo(Forma):
    def __init__(self, cor: str, largura: float,
        altura: float):
        super().__init__(cor)
        self.largura = largura
        self.altura = altura

    def area(self) -> float:
        return self.largura * self.altura

    def perimetro(self) -> float:
        return 2 * (self.largura + self.altura)
```

```
formas: list[Forma] = [
    Circulo("azul", 5),
    Retangulo("vermelho", 4, 3),
    Circulo("verde", 2)
] # Uso polimórfico

for forma in formas:
    print(f"{forma.descricao()}")
    print(f"  Área: {forma.area():.2f}")
    print(f"  Perímetro: {forma.perimetro():.2f}")
```

Polimorfismo:

- ▶ Lista tipada como Forma
- ▶ Aceita qualquer subclasse
- ▶ Método correto chamado em runtime

Python oferece duas funções para inspecionar hierarquias em tempo de execução:

isinstance(obj, Classe)

- ▶ Verifica se obj é instância de Classe ou de qualquer subclasse dela
- ▶ Aceita tupla: isinstance(f, (Circulo, Retangulo))

issubclass(Filha, Pai)

- ▶ Verifica relação entre **classes** (não objetos)
- ▶ Útil em metaprogramação e frameworks

Python

```
formas = [Circulo("azul", 5), Retangulo("vermelho", 4, 3)]

for f in formas:
    # isinstance: verifica se o OBJETO pertence a uma classe
    print(isinstance(f, Forma))      # True (ambos)
    print(isinstance(f, Circulo))    # True apenas para o circulo

    # Util para comportamento especifico
    if isinstance(f, Circulo):
        print(f"Raio: {f.raio}")

# issubclass: verifica se a CLASSE herda de outra
print(issubclass(Circulo, Forma))   # True
print(issubclass(Circulo, Retangulo))# False
print(issubclass(Forma, ABC))       # True
```

Cuidado: Uso excessivo de isinstance pode indicar que o polimorfismo não está bem aplicado. Prefira chamar métodos da classe base.

Objetivo: Refatorar o sistema de recarga utilizando classes abstratas e herança para suportar diferentes tecnologias de carregamento.

1. Classe Abstrata (Carregador):

- ▶ Atributos: nome, potencia (kW).
- ▶ Método abstrato: `calcular_tempo(v: Veiculo)`.

2. Subclasses Concretas:

- ▶ **CarregadorAC:** Implementa o cálculo padrão de tempo.
- ▶ **CarregadorDC:** Aplica um fator de eficiência de 90

Dica para o cálculo em DC:

Python

```
tempo = energia_necessaria / (potencia * 0.9)
```

Desafio: Crie uma lista polimórfica contendo um `CarregadorAC` e um `CarregadorDC`. Calcule e exiba o tempo necessário para carregar o mesmo veículo em ambos os carregadores.

```
from abc import ABC, abstractmethod

class Carregador(ABC):
    def __init__(self, nome: str, potencia: float):
        self.nome = nome
        self.potencia = potencia

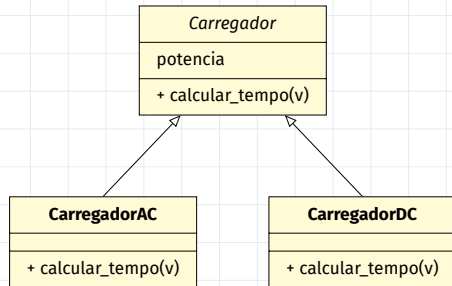
    @abstractmethod
    def calcular_tempo(self, v: Veiculo) -> float:
        pass

class CarregadorAC(Carregador):
    def calcular_tempo(self, v: Veiculo) -> float:
        energia = v.capacidade * (1 - v.carga_atual / 100)
        return energia / self.potencia

class CarregadorDC(Carregador):
    def calcular_tempo(self, v: Veiculo) -> float:
        energia = v.capacidade * (1 - v.carga_atual / 100)
        return energia / (self.potencia * 0.9)

# Uso Polimórfico
v1 = Veiculo("Tesla M3", 75, 20)
carregadores = [CarregadorAC("Wallbox", 7),
                 CarregadorDC("Supercharger", 150)]

for c in carregadores:
    print(f"{c.nome}: {c.calcular_tempo(v1):.2f}h")
```



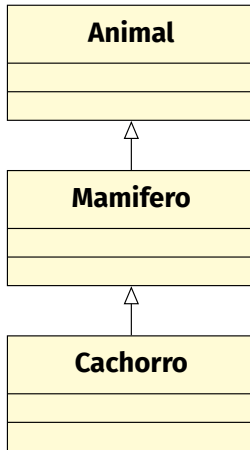
- ▶ **Contrato:** A superclasse obriga a implementação de `calcular_tempo`.
- ▶ **Extensibilidade:** Novos tipos (ex: Indutivo) podem ser adicionados sem alterar o loop principal.



Herança Múltipla



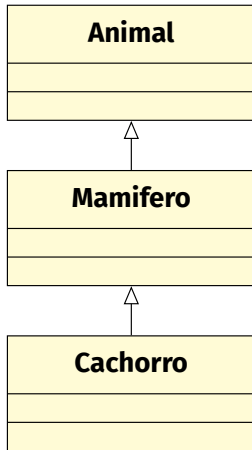
Herança Simples



Uma classe herda de **apenas uma** superclasse.

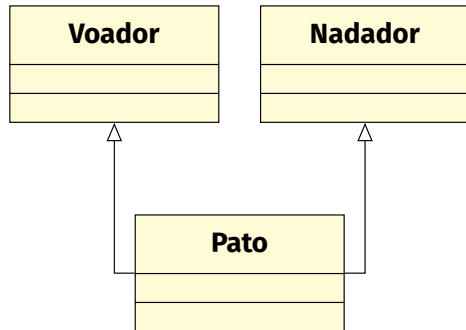
Linguagens: Java, C#

Herança Simples

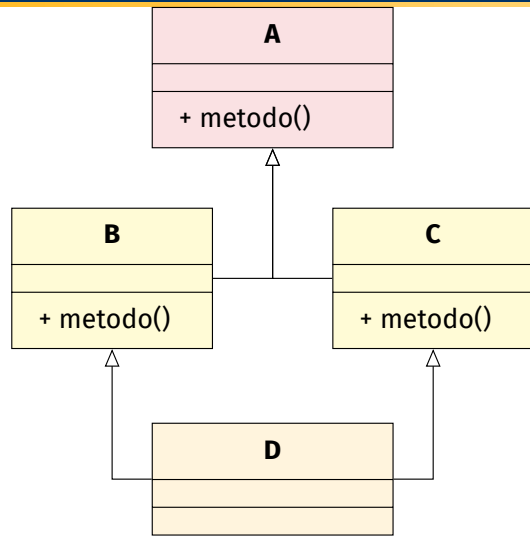


Uma classe herda de **apenas uma** superclasse.
Linguagens: Java, C#

Herança Múltipla



Uma classe herda de **múltiplas** superclasses.
Linguagens: Python, C++



Qual metodo () D herda?

Problema:

- ▶ D herda de B e C
- ▶ Ambos herdam de A
- ▶ `A.metodo()` é chamado duas vezes?
- ▶ Qual versão prevalece?

Solução Python: MRO

- ▶ Method Resolution Order
- ▶ Algoritmo C3 Linearization
- ▶ Ordem determinística e previsível
- ▶ `D.__mro__` mostra a ordem

```
class A:
    def metodo(self):
        print("A.metodo")

class B(A):
    def metodo(self):
        print("B.metodo")
        super().metodo()

class C(A):
    def metodo(self):
        print("C.metodo")
        super().metodo()

class D(B, C): # Ordem importa!
    def metodo(self):
        print("D.metodo")
        super().metodo()
```

MRO de D:

1. D (própria classe)
2. B (primeiro pai)
3. C (segundo pai)
4. A (ancestral comum)
5. object (raiz)

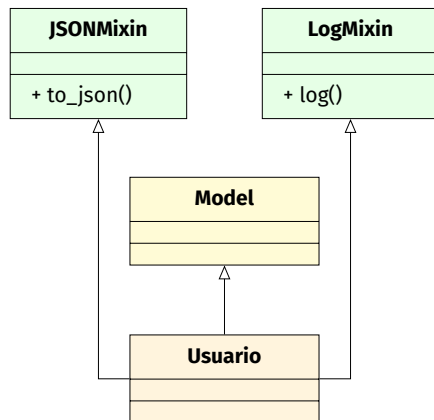
Mixin: Classe projetada para adicionar funcionalidade específica sem ser instanciada sozinha.

Convenções:

- ▶ Nome termina em Mixin
- ▶ Não define `__init__`
- ▶ Não armazena estado próprio
- ▶ Fornece métodos utilitários

Vantagens:

- ▶ Composição de comportamentos
- ▶ Evita hierarquias profundas
- ▶ Reutilização horizontal




```
import json

class JSONMixin:
    """Mixin para serialização JSON"""
    def to_json(self) -> str:
        return json.dumps(self.__dict__)

    @classmethod
    def from_json(cls, data: str):
        return cls(**json.loads(data))

class LogMixin:
    """Mixin para logging"""
    def log(self, msg: str):
        print(f"[{self.__class__.__name__}] {msg}")

class Usuario(JSONMixin, LogMixin):
    def __init__(self, nome: str, email: str):
        self.nome = nome
        self.email = email
```

```
>>>u = Usuario("João", "joao@email.com")
>>>u.log("Usuário criado")
[Usuario] Usuário criado

>>>print(u.to_json())
{"nome": "João", "email": "joao@email.com"}

>>>u2 = Usuario.from_json('{"nome": "Ana",
... "email": "ana@email.com"}')

>>>print(u2.nome)
Ana
```

Observação:

Usuario ganha funcionalidades de ambos os mixins sem herança profunda.

Decorators do Python



Decoradores são funções que recebem outra função como parâmetro, estendem seu comportamento e a retornam (ou a executam).

Funcionamento

- ▶ Baseiam-se no conceito de *Higher-Order Functions*
- ▶ Funcionam como um *Wrapper* (embrulho)
- ▶ Permitem a separação de preocupações (*Cross-cutting concerns*)
- ▶ Modificam a função sem alterar seu código interno

Sintaxe (@)

```
Python @meu_decorador  
def funcao_simples():  
    print("Executando...")
```

O símbolo @ (açúcar sintático) é equivalente a:
`f = meu_decorador(f)`

Aplicações Comuns:

Logging, autenticação, medição de tempo de execução e controle de cache.

O uso de **@property** permite que métodos se comportem como atributos, mantendo o **encapsulamento** sem sacrificar a legibilidade.

Fluxo de Execução

- ▶ **Acesso (Getter):** Quando `c.saldo` é lido, o Python intercepta a chamada e executa o método decorado com `@property`.
- ▶ **Atribuição (Setter):** O comando `c.saldo = 500` dispara o método `@saldo.setter`.
- ▶ **Validação:** A lógica de segurança é executada *antes* da alteração do estado interno.

Estrutura de Memória

- ▶ `_saldo`: Atributo protegido (convenção `_`) onde o dado reside.
- ▶ `saldo`: Nome da interface pública que mascara os métodos.
- ▶ **Read-Only:** Ao omitir o `.setter`, o atributo torna-se imutável externamente.

```
class Conta:
    def __init__(self, titular: str, saldo: float):
        self._titular = titular
        self._saldo = saldo

    @property
    def saldo(self) -> float:
        """Getter - acesso como atributo"""
        return self._saldo

    @saldo.setter
    def saldo(self, valor: float):
        """Setter - validação na atribuição"""
        if valor < 0:
            raise ValueError("Saldo não pode ser negativo")
        self._saldo = valor

    @property
    def titular(self) -> str:
        """Somente leitura (sem setter)"""
        return self._titular
```

```
>>>c = Conta("João", 1000)
>>>c.saldo # Usa getter
1000
>>>c.saldo = 500 # Usa setter
>>>c.saldo = -100 # Dispara exceção
ValueError: Saldo não pode ser negativo

>>>c.titular # Somente leitura
'João'
>>>c.titular = "Maria" # Erro!
AttributeError: property 'titular' has
no setter
```

Vantagens:

- ▶ Acesso como atributo
- ▶ Validação transparente
- ▶ Pode tornar read-only
- ▶ Compatível com código existente

Enquanto métodos comuns operam em **instâncias**, estes decoradores alteram o vínculo do método com a classe.

@classmethod (Factory)

- ▶ **O argumento cls:** Recebe a referência da classe, permitindo instanciar objetos dentro do método.
- ▶ **Padrão Factory:** Métodos como `hoje()` e `from_string()` oferecem formas alternativas de construir o objeto sem sobrecarregar o `__init__`.
- ▶ **Herança:** Se uma subclasse chamar o método, `cls` referenciará a subclasse, garantindo o polimorfismo.

@staticmethod (Utility)

- ▶ **Independência:** Não recebe `self` nem `cls`. Comporta-se como uma função global, mas reside no *namespace* da classe.
- ▶ **Lógica Pura:** Ideal para funções utilitárias que dependem apenas dos parâmetros de entrada, como a lógica de anos bissextos: $ano \pmod{4} = 0$

Tipo de Método	Decorador	1º Argumento	Acesso
Instância	-	<code>self</code>	Atributos do objeto
Classe	<code>@classmethod</code>	<code>cls</code>	Atributos da classe
Estático	<code>@staticmethod</code>	(Nenhum)	Apenas argumentos

```
class Data:
    def __init__(self, dia: int, mes: int, ano: int):
        self.dia = dia
        self.mes = mes
        self.ano = ano

    @classmethod
    def hoje(cls):
        """Factory method - recebe a classe"""
        from datetime import date
        d = date.today()
        return cls(d.day, d.month, d.year)

    @classmethod
    def from_string(cls, data_str: str):
        """Outro factory method"""
        dia, mes, ano = map(int, data_str.split('/'))
        return cls(dia, mes, ano)

    @staticmethod
    def eh_bissexto(ano: int) -> bool:
        """Não precisa de self nem cls"""
        return ano % 4 == 0 and (ano % 100 != 0
        or ano % 400 == 0)
```

```
>>d1 = Data(25, 12, 2024)
>>d2 = Data.hoje() # classmethod
>>d3 = Data.from_string("01/01/2025")

>>Data.eh_bissexto(2024) # staticmethod
True

>>d1.eh_bissexto(2023) # Também funciona
False
```

Diferenças:

@classmethod:

- ▶ Recebe cls (a classe)
- ▶ Útil para factory methods
- ▶ Funciona com herança

@staticmethod:

- ▶ Não recebe self/cls
- ▶ Função utilitária na classe
- ▶ Poderia ser função normal

Python

```
from abc import ABC, abstractmethod

class Repositorio(ABC):
    """Interface para repositórios"""

    @abstractmethod
    def salvar(self, entidade) -> None:
        """Deve ser implementado"""
        pass

    @abstractmethod
    def buscar(self, id: int):
        pass

    @property
    @abstractmethod
    def total(self) -> int:
        """Property abstrata"""
        pass

    @classmethod
    @abstractmethod
    def criar_conexao(cls):
        """Classmethod abstrato"""
        pass
```

Python

```
class RepositorioMemoria(Repositorio):
    def __init__(self):
        self._dados = {}

    def salvar(self, entidade) -> None:
        self._dados[entidade.id] = entidade

    def buscar(self, id: int):
        return self._dados.get(id)

    @property
    def total(self) -> int:
        return len(self._dados)

    @classmethod
    def criar_conexao(cls):
        return cls()
```

Combinações:

- ▶ @property + @abstractmethod
- ▶ @classmethod + @abstractmethod
- ▶ @staticmethod + @abstractmethod

Introduzido no Python 3.7, o **@dataclass** gera automaticamente métodos especiais como `__init__`, `__repr__` e `__eq__`.

Python

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Coordenada:
    x: float
    y: float
```

O parâmetro `frozen=True` torna o objeto imutável, o seu estado (seus dados internos) não pode ser alterado após a sua criação, permitindo que seja usado como chave de dicionário.

```
>>p1 = Coordenada(1.5, 2.0)
>>p1
    Coordenada(x=1.5, y=2.0)

>>p2 = Coordenada(1.5, 2.0)
>>p1 == p2
    True
```

Vantagens:

- ▶ Código extremamente conciso
- ▶ Tipagem estática amigável
- ▶ Reduz erros em métodos de comparação

Por padrão, o decorador analisa as anotações de tipo e gera métodos baseados na estrutura dos dados.

Geração Automática

- ▶ **__init__**: Cria o construtor usando os nomes dos campos na ordem definida.
- ▶ **__repr__**: Gera uma *string* legível para depuração:
Classe(campo=valor).
- ▶ **__eq__**: Implementa comparação por **valor** (campo a campo), ignorando o endereço de memória.

Exemplo Gerado

```
@dataclass
class Produto:
    id: int
    nome: str

# O Python gera o equivalente a:
# def __init__(self, id, nome): ...
# def __repr__(self): ...
# def __eq__(self, other): ...
```

Caso a implementação automática não atenda aos requisitos, pode-se definir o método manualmente. O Python prioriza a sua definição.

Python

```
@dataclass
class Usuario:
    username: str
    email: str

    def __repr__(self):
        """Customizando a representação"""
        return f"User: {self.username} <{self.email}>"

    def __post_init__(self):
        """Lógica extra após o __init__ automático"""
        self.username = self.username.lower()
```

Nota: O `__post_init__` é uma alternativa elegante para adicionar lógica ao construtor sem precisar sobrescrever o `__init__` inteiro.

Comportamento

```
>>u = Usuario("Admin", "a@b.com")
>>u
User: admin <a@b.com>
# Note o username em minúsculo
```

Quando Sobrescrever?

- ▶ Formatação específica de *logs*.
- ▶ Validações complexas no `__init__`.
- ▶ Comparação (`__eq__`) por IDs únicos em vez de todos os campos.

Objetivo: Refatorar a gestão de sensores de uma planta industrial utilizando decoradores para garantir integridade e extensibilidade.

1. Estrutura Base:

- ▶ Crie uma classe abstrata `SensorBase` com uma `@property` abstrata chamada `id`.
- ▶ Implemente `Sensor` como uma `@dataclass` herdando de `SensorBase`.

2. Lógica e Decoradores:

- ▶ Use `@property` para o atributo `leitura`, impedindo valores negativos no setter.
- ▶ Crie um `@classmethod` para instanciar o sensor a partir de uma *string* formatada.
- ▶ Use `@staticmethod` para validar se uma unidade de medida é aceita.

Desafio: O sensor deve ser imutável após a criação (frozen), mas o valor da leitura deve ser validado no momento da atribuição inicial. Como conciliar `@dataclass` com validação de propriedades?



```
from dataclasses import dataclass
from abc import ABC, abstractmethod
class SensorBase(ABC):
    @property
    @abstractmethod
    def id(self): pass
@dataclass
class Sensor(SensorBase):
    _id: str
    _leitura: float

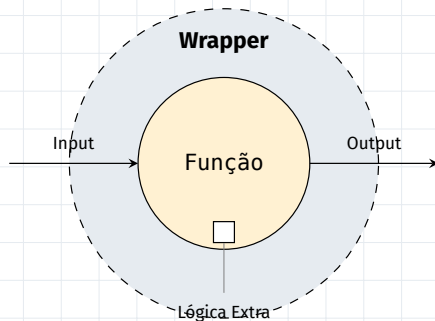
    @property
    def id(self): return self._id

    @property
    def leitura(self): return self._leitura

    @leitura.setter
    def leitura(self, valor):
        if valor < 0: raise ValueError("Leitura negativa")
        self._leitura = valor

    @classmethod
    def de_string(cls, dados: str):
        id_s, valor = dados.split(';') # Exemplo: "S01;25.5"
        return cls(id_s, float(valor))

    @staticmethod
```



- ▶ **Encapsulamento:** O uso de `_id` protege o estado interno enquanto a `@property` expõe a interface.
- ▶ **Eficiência:** O `@classmethod` atua como uma fábrica (*factory*), simplificando a criação de objetos

Override (Sobrescrita)

Python

```
class Animal:
    def falar(self):
        print("Som genérico")

class Cachorro(Animal):
    def falar(self): # Override
        print("Au Au!")

class Gato(Animal):
    def falar(self): # Override
        print("Miau!")
```

- ▶ Mesmo nome, mesma assinatura
- ▶ Subclasse redefine comportamento
- ▶ Polimorfismo em runtime

Overload (Sobrecarga)

Python NÃO tem overload nativo!

Python

```
class Calculadora:
    # Forma PYTHONICA: valores default e *args
    def soma(self, a, b=0, *extras):
        return a + b + sum(extras)

c = Calculadora()
c.soma(1, 2) # 3
c.soma(1, 2, 3, 4) # 10

# typing.overload: APENAS documentacao para mypy
# NAO muda o comportamento em runtime!
from typing import overload

class Formato:
    @overload
    def parse(self, v: int) -> str: ...
    @overload
    def parse(self, v: str) -> int: ...
    def parse(self, v): # unica implementacao
        real
        return str(v) if isinstance(v, int) else
        int(v)
```

Mãos à obra



Objetivo: Implementar a lógica completa de um torniquete utilizando o padrão de projeto *State*.

1. Contexto (Torniquete):

- ▶ Deve possuir um atributo privado `_estado`.
- ▶ Métodos `moeda()` e `empurrar()` que delegam a ação para o estado atual.
- ▶ Uma *property* para ler o nome do estado.

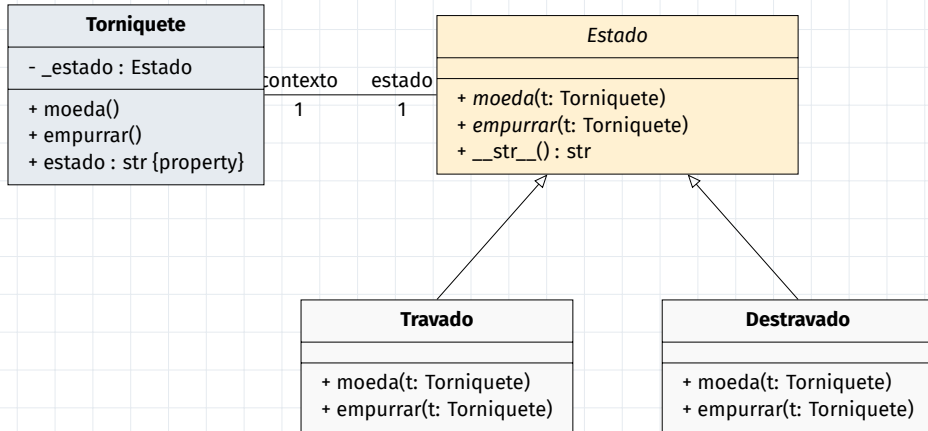
2. Estados:

- ▶ **Travado:** Libera com moeda; emite alarme ao ser empurrado.
- ▶ **Destravado:** Ignora moeda (devolve); trava após ser empurrado.

Regra de Ouro: O contexto (Torniquete) não deve conter nenhum bloco `if` ou `match` para verificar o estado atual.



Modelagem: Padrão State para o Torniquete



- ▶ A associação indica que o Torniquete "tem-um" Estado.
- ▶ As subclasses concretas definem o comportamento específico de cada situação.



```
from abc import ABC, abstractmethod

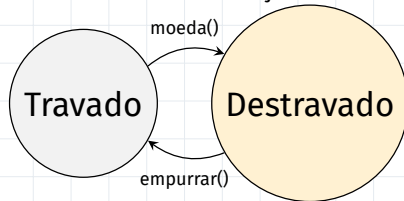
class Estado(ABC):
    @abstractmethod
    def moeda(self, t: 'Torniquete'): pass
    @abstractmethod
    def empurrar(self, t: 'Torniquete'): pass
    def __str__(self): return self.__class__.__name__

class Travado(Estado):
    def moeda(self, t):
        print("Destravando..."); t._estado = Destravado()
    def empurrar(self, t):
        print("Alarme: Acesso negado!")

class Destravado(Estado):
    def moeda(self, t):
        print("Moeda devolvida.")
    def empurrar(self, t):
        print("Passagem liberada."); t._estado = Travado()

class Torniquete:
    def __init__(self): self._estado = Travado()
    @property
    def estado(self): return str(self._estado)
    def moeda(self): self._estado.moeda(self)
    def empurrar(self): self._estado.empurrar(self)
```

Fluxo de Transição



- ▶ **Desacoplamento:** O torniquete não conhece as regras de transição.
- ▶ **Extensibilidade:** Para adicionar um estado "Quebrado", basta criar uma nova classe.



▶ Download .py

Hierarquia e Estrutura

- ▶ **Sintaxe:** `class Filha(Pai)` define a relação de especialização.
- ▶ **Acesso à Base:** `super()` é essencial para garantir a execução da lógica da superclasse.
- ▶ **Abstração:** Uso de ABC e `@abstractmethod` para definir interfaces obrigatórias.
- ▶ **Múltipla Herança:** Resolução via **MRO** e uso de **Mixins** para composição de funcionalidades.

Ferramentas e Decoradores

- ▶ **@property:** Implementa *getters* e *setters* de forma elegante (encapsulamento).
- ▶ **@classmethod:** Ideal para *factory methods* (recebe `cls`).
- ▶ **@staticmethod:** Funções utilitárias sem vínculo com estado de instância ou classe.
- ▶ **@dataclass:** Geração automática de `__init__`, `__repr__` e `__eq__`.
- ▶ **__post_init__:** Gancho para lógica extra após a inicialização automática.

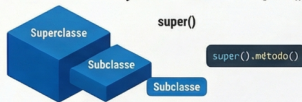
Regra de Ouro

Priorize a legibilidade: use herança para relações "é um" e decoradores para estender comportamentos sem modificar o código original.

Guia Essencial: Herança e Decoradores em Python

Pilares da Herança e Polimorfismo

Relação "É-Um" e super()



A herança permite que subclasses recebam atributos e métodos de uma superclasse, usando `super()` para acessar a base.

	Tipo de Classe	Pode ser Instanciada?	Implementação de Métodos
1	Concreta	✓ Sim	Todos os métodos implementados
2	Abstrata	✗ Não	Pode ter métodos abstratos e concretos
3	Interface	✗ Não	Apenas métodos abstratos (contrato puro)

Polimorfismo via Sobrescrita (Override)



Subclasses podem redefinir métodos da classe pai para especializar comportamentos, mantendo uma interface uniforme.

Ordem de Resolução de Métodos (MRO)



CS Linearization

Em herança múltipla, o Python usa o algoritmo C3 Linearization para determinar a ordem de execução.

O Poder dos Decoradores

@property para Encapsulamento



Permite que métodos funcionem como atributos, facilitando validações transparentes em getters e setters.

@classmethod vs @staticmethod



Classmethods recebem a referência de classe (Factory); **staticmethods** são funções utilitárias sem acesso à classe/instância.

Eficiência com @dataclass



Gera automaticamente métodos especiais (`__init__`, `__repr__`, `__eq__`), reduzindo drasticamente o código repetitivo (boilerplate).



Prof. Edson José Pacheco

pacheco@ufpr.br

UFPR

UNIVERSIDADE FEDERAL DO PARANÁ